Chapter 08: Naive Bayes Classifier

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [1]:
```
import matplotlib.pyplot as plt
import mlba
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB # the Naive Bayes classifier
from sklearn.pipeline import FunctionTransformer, Pipeline # Pipeline allows
from sklearn.preprocessing import OneHotEncoder
```

```
no display found. Using non-interactive Agg backend
```

Example 3: Predicting Delayed Flights

Predicting flight delays can be useful to a variety of organizations: airport authorities, airlines, and aviation authorities. At times, joint task forces have been formed to address the problem. If such an organization were to provide ongoing real-time assistance with flight delays, it would benefit from some advance notice about flights that are likely to be delayed. In this simplified illustration, we look at five predictors (see Table 8.3). The outcome of interest is whether or not the flight is delayed (delayed here means arrived more than 15 minutes late). Our data consist of all flights from the Washington, DC area into the New York City area during January 2004. A record is a particular flight. The percentage of delayed flights among these 2201 flights is 19.5%. The data were obtained

from the Bureau of Transportation Statistics (available on the web at www.transtats.bts.gov). The goal is to accurately predict whether or not a new flight (not in this dataset) will be delayed. The outcome variable is whether the flight was delayed, and thus it has two classes (1 = delayed and 0 = on time). In addition, information is collected on the predictors listed in Table 8.3.

Table 8.4 Naive Bayes Classifier Applied to Flight Delays (Training) Data

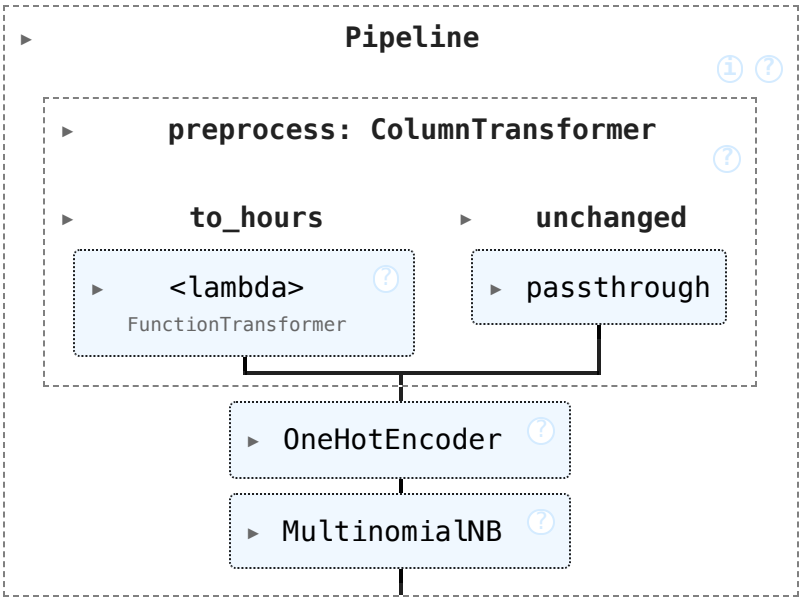
After converting all predictors to categorical and creating dummies, the data were partitioned into training (60%) and holdout (40%) sets, and then a naive Bayes classifier was applied to the training set.

In [10]:
```
#VB67
#loading the dataset
delays_df = mlba.load_data('FlightDelays.csv')

predictors = ['DAY_WEEK', 'CRS_DEP_TIME', 'ORIGIN', 'DEST', 'CARRIER']
outcome = 'Flight Status'
delays_df[outcome] = delays_df[outcome].astype('category') # convert outcome

# splitting into training and holdout
train_X, holdout_X, train_y, holdout_y = train_test_split(
    delays_df[predictors], delays_df[outcome],
    test_size=0.40, random_state=1)

preprocess = ColumnTransformer([
    # create hourly bins departure time to reduce the number of categories
    ('to_hours', FunctionTransformer(lambda x: round(x / 100).astype('int'))
                 ['CRS_DEP_TIME']),
    # keep all other features as they are
    ('unchanged', 'passthrough', ['DAY_WEEK', 'ORIGIN', 'DEST', 'CARRIER']),
])

# Define a complete modeling pipeline
delays_nb = Pipeline([
    ('preprocess', preprocess), # applies time conversion and passthrough
    ('encoder', OneHotEncoder()), # encodes all categorical columns
    ('nb', MultinomialNB(alpha=0.01)), # trains Naive Bayes
])
delays_nb.fit(train_X, train_y) # fit the pipeline to the training data

# predict class probabilities on training and holdout sets
predProb_train = delays_nb.predict_proba(train_X)
predProb_holdout = delays_nb.predict_proba(holdout_X)

# predict actual class labels (delayed / on-time) on both sets
holdout_y_pred = delays_nb.predict(holdout_X)
train_y_pred = delays_nb.predict(train_X)
```

The pipeline may look overly complex, but it is fairly straightforward when you think through the individual steps. The full model definition is shown when printing the pipeline object.

In [4]:
```
#VB67
delays_nb
```

Out[4]:



The `preprocess` step transforms the `CRS_DEP_TIME` variable into hourly bins. We define the transformation using a lambda function `lambda x round(x / 100).astype('int')`. This is just a shorthand to make the code more concise. Following the preprocess step all features are categorical and we use the `OneHotEncoder` to convert them into a binary format in the `encoder` step. After this, we have the data in the format we need for the `MultinomialNB` model (step `nb`).

Table 8.5 Pivot Table of Flight Status by Destination Airport (Training Data)

Before using the output, let's see how the algorithm works. We start by generating pivot tables for the outcome vs. each of the five predictors using the training set, in order to obtain conditional probabilities.

In [6]:
```
#VB67
# splitting the original data frame into a train and test using the same ran
train_df, holdout_df = train_test_split(delays_df, test_size=0.4, random_sta

# set display precision for float numbers
pd.set_option('display.precision', 4)

# printing prior probabilities for each class
print(train_df['Flight Status'].value_counts() / len(train_df))
print()
```

```
# looping over each predictor variable
for predictor in predictors:
    # make a copy of the two columns needed (the outcome and the predictor)
    df = train_df[['Flight Status', predictor]].copy()

    # for departure time, convert it to round hour
    if predictor == 'CRS_DEP_TIME':
        df[predictor] = (df[predictor] / 100).round().astype(int)

    # create a frequency table: how often each predictor value occurs for ea
    freqTable = df.pivot_table(index='Flight Status', columns=predictor, agg
                               observed=False)

    # divide each value by the sum of the row to get conditional probabiliti
    propTable = freqTable.apply(lambda x: x / sum(x), axis=1)

    print(propTable)
    print()
# reset display settings
pd.reset_option('display.precision')
```

```
Flight Status
ontime     0.8023
delayed    0.1977
Name: count, dtype: float64

DAY_WEEK           1       2       3       4       5      6       7
Flight Status
delayed        0.1916  0.1494  0.1149  0.1264  0.1877  0.069  0.1609
ontime         0.1246  0.1416  0.1445  0.1794  0.1690  0.136  0.1048

CRS_DEP_TIME       6       7       8       9       10      11      12      1
3  \
Flight Status
delayed        0.0345  0.0536  0.0651  0.0192  0.0307  0.0115  0.0498  0.046
0
ontime         0.0623  0.0633  0.0850  0.0567  0.0519  0.0340  0.0661  0.074
6

CRS_DEP_TIME       14      15      16      17      18      19      20      2
1
Flight Status
delayed        0.0383  0.2031  0.0728  0.1533  0.0192  0.0996  0.0153  0.088
1
ontime         0.0576  0.1171  0.0774  0.1001  0.0349  0.0397  0.0264  0.052
9

ORIGIN            BWI     DCA     IAD
Flight Status
delayed        0.0805  0.5211  0.3985
ontime         0.0604  0.6478  0.2918

DEST              EWR     JFK     LGA
Flight Status
delayed        0.3793  0.1992  0.4215
ontime         0.2663  0.1558  0.5779

CARRIER            CO      DH      DL      MQ      OH      RU      UA      U
S
Flight Status
delayed        0.0575  0.3142  0.0958  0.2222  0.0077  0.2184  0.0153  0.069
0
ontime         0.0349  0.2295  0.2040  0.1171  0.0104  0.1690  0.0170  0.218
1
```

Table 8.6 Scoring the Example Flight (Probability and Class)

In [8]:
```
#VB67
# Classifying a Delta flight from DCA to LGA departing between 10:00 AM and
df = pd.DataFrame({
    'CARRIER': ['DL'],
    'ORIGIN': ['DCA'],
    'DEST': ['LGA'],
```

```
    'CRS_DEP_TIME': [1000],
    'DAY_WEEK': [7],
})

# Defining predictors
predictors = ['DAY_WEEK', 'CRS_DEP_TIME', 'ORIGIN', 'DEST', 'CARRIER']

# Predicting class and probabilities
predicted_class = delays_nb.predict(df[predictors])[0]
predicted_probs = delays_nb.predict_proba(df[predictors])[0]

print(f'Class: {predicted_class}, Probabilities: {predicted_probs}')
```

```
Class: ontime, Probabilities: [0.05798946 0.94201054]
```

Table 8.7 Confusion Matrices for Flight Delay Using the Naive Bayes Classifier

In [9]:

```
#VB67
# training
mlba.classificationSummary(y_true=train_y, y_pred=train_y_pred) #, class_nam

# holdout
mlba.classificationSummary(y_true=holdout_y, y_pred=holdout_y_pred) #, class
```

```
Confusion Matrix (Accuracy 0.7955)

        Prediction
 Actual delayed  ontime
delayed      52     209
 ontime      61     998
Confusion Matrix (Accuracy 0.7821)

        Prediction
 Actual delayed  ontime
delayed      26     141
 ontime      51     663
```

In [ ]:

```

```